

Universitatea Transilvania din Braşov  
Facultatea de Matematică şi Informatică  
Specializarea: Informatică Aplicată

---

**Raport Experimental**  
**Cuantizare Numerică FP16 şi INT8**  
**pe Vision Transformer Pretrained**  
(ImageNet-1k / ImageNette)

---

**Student:**

Tudose Alexandru  
Grupa 10LF333  
An III, Informatică Aplicată

**Coordonator:**

Conf. dr. ing. Honorius Gâlmeanu  
Departamentul de Informatică

Braşov, 2026

# Cuprins

|          |                                                            |           |
|----------|------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introducere și Motivație</b>                            | <b>2</b>  |
| <b>2</b> | <b>Arhitectura Modelului</b>                               | <b>2</b>  |
| 2.1      | Vision Transformer — ViT-Tiny . . . . .                    | 2         |
| 2.2      | Straturile cuantizabile . . . . .                          | 3         |
| 2.3      | Preprocessing corect . . . . .                             | 3         |
| <b>3</b> | <b>Dataset și Configurație Experimentală</b>               | <b>3</b>  |
| 3.1      | ImageNette2-320 . . . . .                                  | 3         |
| <b>4</b> | <b>Faza 1 — Cuantizare FP16 Statică</b>                    | <b>4</b>  |
| 4.1      | Metodologie . . . . .                                      | 4         |
| 4.2      | Rezultate . . . . .                                        | 4         |
| 4.3      | Discuție . . . . .                                         | 5         |
| <b>5</b> | <b>Faza 2 — Cuantizare INT8 per-tensor (weight-only)</b>   | <b>5</b>  |
| 5.1      | Metodologie . . . . .                                      | 5         |
| 5.2      | Straturi cuantizate . . . . .                              | 6         |
| 5.3      | Rezultate . . . . .                                        | 6         |
| 5.4      | Analiză per-layer . . . . .                                | 7         |
| 5.5      | Motivul lipsei speedup-ului . . . . .                      | 7         |
| <b>6</b> | <b>Comparație Globală: FP32 / FP16 / INT8</b>              | <b>8</b>  |
| <b>7</b> | <b>Analiză și Discuții</b>                                 | <b>9</b>  |
| 7.1      | FP16 — conversie fără cost . . . . .                       | 9         |
| 7.2      | INT8 weight-only — compresie maximă fără speedup . . . . . | 9         |
| 7.3      | Implicații pentru deployment . . . . .                     | 9         |
| <b>8</b> | <b>Limitări ale studiului</b>                              | <b>9</b>  |
| <b>9</b> | <b>Concluzii și Pași Următori</b>                          | <b>10</b> |
| 9.1      | Concluzii . . . . .                                        | 10        |
| 9.2      | Faza 3 — Experimente planificate . . . . .                 | 10        |

# 1 Introducere și Motivație

Prezentul raport documentează rezultatele experimentelor din **Fazele 1 și 2** ale lucrării de licență, ce studiază impactul cuantizării numerice asupra unui Vision Transformer (ViT). Spre deosebire de studiul preliminar (antrenare pe CIFAR-10), aceste faze utilizează un model **pretrained pe ImageNet-1k** și evaluează cuantizarea exclusiv la inferență (*post-training quantization*, *PTQ*), fără nicio re-antrenare.

## Corecții față de studiul preliminar:

- Implementarea anterioară de FP8 folosea cast direct `torch.float8_e4m3fn` — problematică deoarece nu toate valorile FP32 sunt reprezentabile. Faza 2 implementează cuantizarea corectă: scalare liniară cu `int8`, garantând o mapare bijectivă la 256 valori discrete.
- LayerNorm și classification head sunt excluse de la cuantizare (sensibile la erori numerice).
- Modelul este acum evaluat pe **ImageNette2-320**, un subset de 10 clase din ImageNet-1k, cu ponderile originale pretrained — fără fine-tuning.

## Întrebări de cercetare:

1. Cuantizarea statică FP16 (`model.half()`) degradează acuratețea?
2. Cuantizarea INT8 weight-only per-tensor introduce degradare semnificativă?
3. Ce layere sunt cele mai sensibile la cuantizare (MSE per layer)?
4. Care sunt câștigurile reale de memorie și latență pe Apple M4 (MPS)?

# 2 Arhitectura Modelului

## 2.1 Vision Transformer — ViT-Tiny

Modelul utilizat este `vit_tiny_patch16_224` din biblioteca `timm`, pretrained pe ImageNet-1k cu 5.7 milioane de parametri. Arhitectura include:

**Tabela 1:** Parametrii arhitecturali ai ViT-Tiny

| Proprietate          | Valoare                                |
|----------------------|----------------------------------------|
| Parametri totali     | 5.7 milioane                           |
| Patch size           | $16 \times 16$ pixeli                  |
| Input resolution     | $224 \times 224$                       |
| Embedding dim        | 192                                    |
| Număr blocuri        | 12                                     |
| Număr capete (MHA)   | 3                                      |
| Pre-training dataset | ImageNet-1k (1.2M imagini, 1000 clase) |

Blocul central Transformer operează cu Multi-Head Self-Attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V \quad (1)$$

unde  $Q, K, V \in \mathbb{R}^{N \times d_k}$  sunt proiecțiile query, key și value. Fiecare bloc include și o rețea feed-forward (MLP):

$$\text{FFN}(x) = \text{GELU}(xW_1 + b_1)W_2 + b_2 \quad (2)$$

Ambele sub-straturi sunt înconjurate de conexiuni reziduale și **LayerNorm** — acesta din urmă *nu* este cuantizat în Faza 2 (sensibil la erori numerice).

## 2.2 Straturile cuantizabile

Din totalul de 48 straturi `nn.Linear` identificate:

- `blocks.N.attn.qkv` (12 straturi) — proiecție Q, K, V combinată
- `blocks.N.attn.proj` (12 straturi) — proiecție output attention
- `blocks.N.mlp.fc1` (12 straturi) — prima proiecție MLP
- `blocks.N.mlp.fc2` (12 straturi) — a doua proiecție MLP

Exclude: `norm` (LayerNorm), `cls_token`, `pos_embed`, `head` (classification head).

## 2.3 Preprocessing corect

**Notă importantă:** `timm` folosește pentru `vit_tiny_patch16_224` normalizare cu  $\mu = \sigma = 0.5$  (nu normalizarea ImageNet standard  $\mu = [0.485, 0.456, 0.406]$ ). Utilizarea normalizării greșite a redus acuratețea de la  $\approx 79\%$  la  $47\%$ . Configul corect a fost obținut via:

```
data_config = timm.data.resolve_data_config({}, model=model)
val_transform = timm.data.create_transform(**data_config, is_training=False)
```

# 3 Dataset și Configurație Experimentală

## 3.1 ImageNette2-320

ImageNette [5] este un subset de 10 clase din ImageNet-1k, ales ca proxy pentru evaluare rapidă locală (fără a necesita descărcarea celor 150 GB ai ImageNet complet).

**Tabela 2:** Configurație experiment

| Parametru        | Valoare                                      |
|------------------|----------------------------------------------|
| Model            | <code>vit_tiny_patch16_224</code>            |
| Pre-training     | ImageNet-1k ( <code>pretrained=True</code> ) |
| Dataset evaluare | ImageNette2-320 (10 clase, 3925 imagini val) |
| Device           | Apple M4 MPS                                 |
| Batch size       | 64                                           |
| PyTorch          | 2.9.1                                        |

**Maparea label-urilor:** Modelul pretrained produce logits pentru 1000 clase ImageNet-1k. Folderele ImageNette (WordNet IDs) sunt mapate la indicii corecți din ImageNet-1k (ex. `n01440764` → clasa 0 “tench”).

## 4 Faza 1 — Cuantizare FP16 Statică

### 4.1 Metodologie

Faza 1 evaluează conversia completă a ponderilor la float16 prin `model.half()`, urmată de inferență pe MPS:

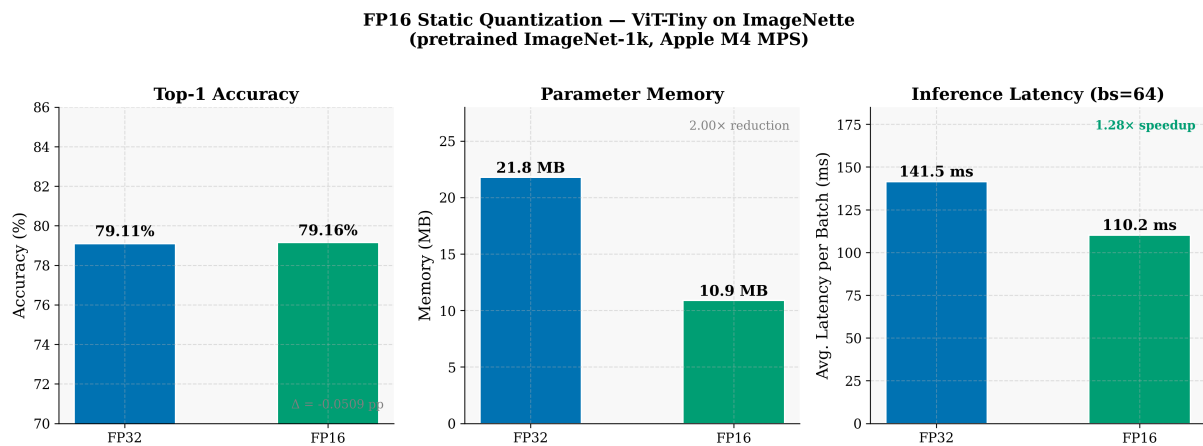
```
model_fp16 = timm.create_model("vit_tiny_patch16_224",
                               pretrained=True, num_classes=1000)
model_fp16 = model_fp16.half().to(device).eval()
```

Aceasta este o cuantizare **statică** (nu AMP/autocast) — *toate* ponderile sunt stocate și procesate în float16, inclusiv LayerNorm.

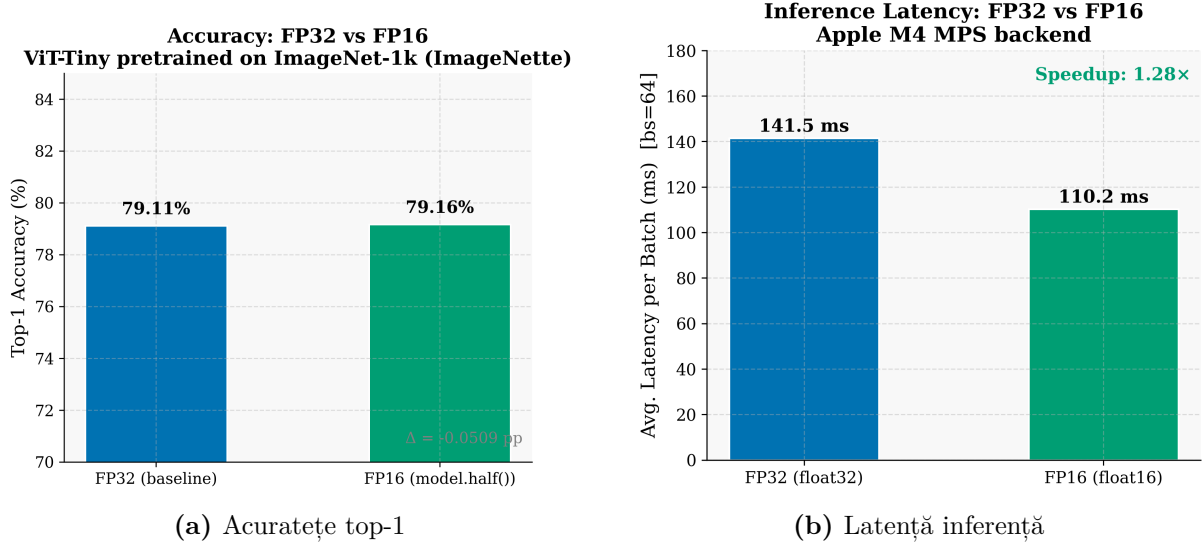
### 4.2 Rezultate

**Tabela 3:** Rezultate Faza 1 — FP32 vs. FP16 pe ImageNette

| Metodă                             | Acuratețe | Loss   | Latență (ms/batch)     | Memorie (MB)         |
|------------------------------------|-----------|--------|------------------------|----------------------|
| FP32 (baseline)                    | 79.11%    | 0.7474 | 141.45                 | 21.81                |
| FP16 ( <code>model.half()</code> ) | 79.16%    | 0.7472 | 110.25                 | 10.91                |
| <b>Diferență</b>                   | −0.051 pp | —      | <b>1.28×</b> mai rapid | <b>2.00×</b> mai mic |



**Figura 1:** Overview Faza 1: comparație FP32 vs. FP16 pe cele trei metrice principale. Acuratețea este practic identică ( $\Delta = -0.051$  pp), memoria se reduce exact  $2\times$  (4 bytes  $\rightarrow$  2 bytes per parametru), latența scade cu  $1.28\times$ .



**Figura 2:** Detaliu acuratețe și latență. FP16 este marginal *mai bun* cu  $-0.051$  pp (zgomot numeric, nu câștig real), și  $1.28\times$  mai rapid pe MPS Apple M4.

### 4.3 Discuție

**Acuratețe:** Degradarea de  $-0.051$  pp (FP16 marginal mai bun) se încadrează în variabilitatea numerică și nu poate fi atribuită preciziei. ViT-Tiny este robust la conversia `float32`  $\rightarrow$  `float16`.

**Memorie:** Reducerea exactă de  $2\times$  ( $21.81$  MB  $\rightarrow$   $10.91$  MB) confirmă teoria: `float16` ocupă exact jumătate față de `float32`.

**Latență:** Speedup-ul de  $1.28\times$  pe Apple M4 MPS este real, dar mai modest decât pe GPU NVIDIA (unde Tensor Cores pentru FP16 oferă  $1.5\text{--}3\times$ ). Apple Silicon valorifică `float16` nativ, dar fără unități dedicate.

**Compatibilitate MPS:** `model.half()` funcționează complet pe MPS cu PyTorch 2.9, inclusiv LayerNorm în `float16` — nu a fost necesar fallback pe CPU.

## 5 Faza 2 — Cuantizare INT8 per-tensor (weight-only)

### 5.1 Metodologie

Faza 2 implementează cuantizarea corectă pe 8 biți cu scalare liniară (*weight-only, per-tensor*):

$$s = \frac{\max(|W|)}{127}, \quad q = \text{clamp}\left(\text{round}\left(\frac{W}{s}\right), -128, 127\right) \in \mathbb{Z}_8 \quad (3)$$

La inferență, ponderile sunt decuantizate înainte de multiplicare matriceală:

$$\hat{W} = q \cdot s \in \mathbb{R} \quad (4)$$

Implementarea utilizează un wrapper `QuantizedLinear` care stochează (`weight_int8`, `scale`) și dequantizează la fiecare forward pass.

## 5.2 Straturi cuantizate

**Tabela 4:** Strategia de cuantizare selectivă

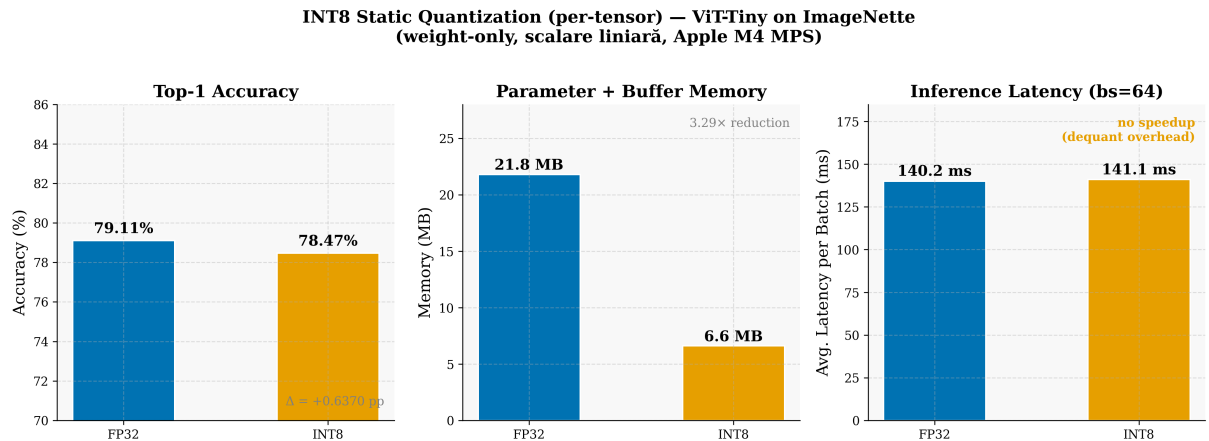
| Tip strat            | Cuantizat? | Motivație                  |
|----------------------|------------|----------------------------|
| blocks.N.attn.qkv    | Da         | matmul, cuantizabil        |
| blocks.N.attn.proj   | Da         | matmul, cuantizabil        |
| blocks.N.mlp.fc1     | Da         | matmul, cuantizabil        |
| blocks.N.mlp.fc2     | Da         | matmul, cuantizabil        |
| LayerNorm (norm)     | <b>Nu</b>  | sensibil la erori numerice |
| head (classifier)    | <b>Nu</b>  | strat final, impact direct |
| cls_token, pos_embed | <b>Nu</b>  | embeddings, nu Linear      |

**Total:** 48 straturi nn.Linear cuantizate, 1 strat exclus (head).

## 5.3 Rezultate

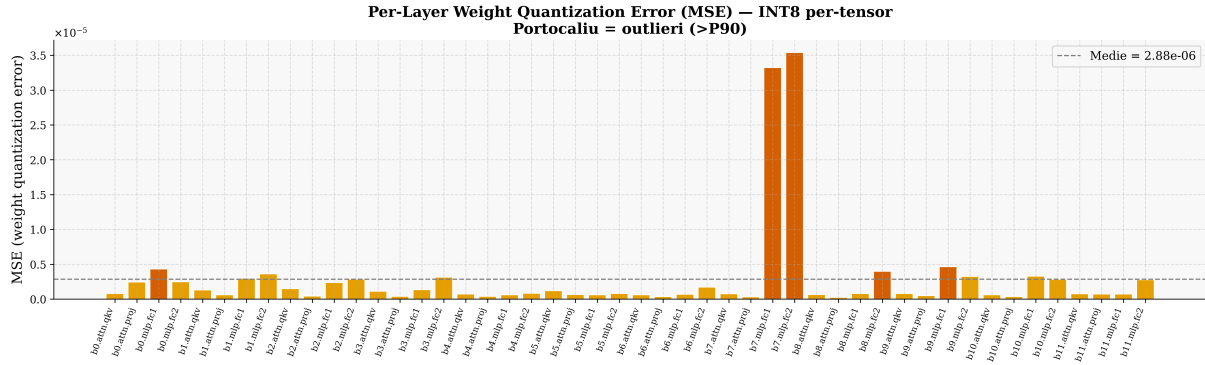
**Tabela 5:** Rezultate Faza 2 — FP32 vs. INT8 pe ImageNette

| Metodă            | Acuratețe        | Loss          | Latență (ms/batch)                     | Memorie (MB)                           |
|-------------------|------------------|---------------|----------------------------------------|----------------------------------------|
| FP32 (baseline)   | 79.11%           | 0.7474        | 140.16                                 | 21.81                                  |
| INT8 (per-tensor) | 78.47%           | 0.7712        | 141.12                                 | 6.62                                   |
| <b>Diferență</b>  | <b>+0.637 pp</b> | <b>+0.024</b> | <b><math>\approx 1.00\times</math></b> | <b><math>3.29\times</math> mai mic</b> |

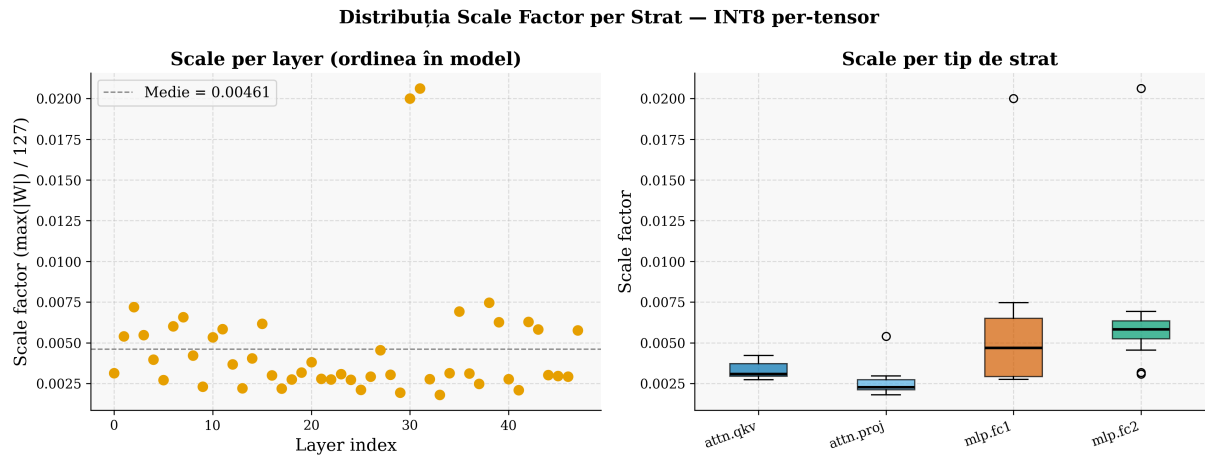


**Figura 3:** Overview Faza 2: comparație FP32 vs. INT8. Degradarea de +0.637 pp este acceptabilă pentru weight-only INT8. Reducerea de memorie este  $3.29\times$  (biasul rămâne în float32). Nu există speedup real deoarece dequantizarea la fiecare forward pass anulează câștigul de bandwidth.

## 5.4 Analiză per-layer



**Figura 4:** MSE al erorii de cuantizare per strat (eroarea  $\|W - \hat{W}\|_F^2/n$ ). Cele mai multe straturi au MSE în intervalul  $[3 \times 10^{-7}, 5 \times 10^{-6}]$ . Straturile `blocks.7.mlp.fc1` și `blocks.7.mlp.fc2` sunt **outlieri** cu MSE de  $\approx 40\times$  mai mare decât media — cauzat de ponderi cu valori extreme (scale factor  $\approx 0.02$ , față de media  $\approx 0.004$ ). Acestea sunt candidați pentru cuantizare per-channel în Faza 3.



**Figura 5:** *Stânga:* Factorul de scalare  $s = \max(|W|)/127$  pentru fiecare strat, în ordinea din model. Variabilitate ridicată în straturile finale ale blocului 7. *Dreapta:* Distribuție boxplot per tip de strat. Straturile `mlp.fc1` și `mlp.fc2` prezintă cea mai mare variabilitate a scale-ului, indicând că granularitatea per-tensor este insuficientă pentru acestea.

## 5.5 Motivul lipsei speedup-ului

Deși ponderile sunt stocate în `int8` (1 byte), dequantizarea la `float32` înainte de fiecare matmul introduce un overhead care compensează câștigul de bandwidth. Speedup-ul real apare doar cu:

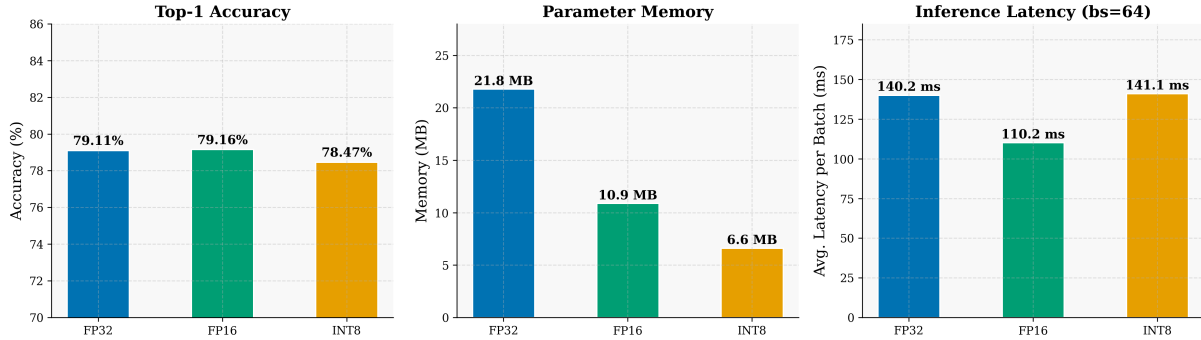
- Kernel-uri dedicate INT8 GEMM (ex. `torch.int_mm` pe CUDA)
- Hardware cu suport nativ INT8 (NVIDIA A100, H100 – Tensor Cores)
- Activări cuantizate simultan (*weight + activation quantization*)

Pe MPS Apple M4, nu există suport nativ INT8 GEMM în PyTorch 2.9.



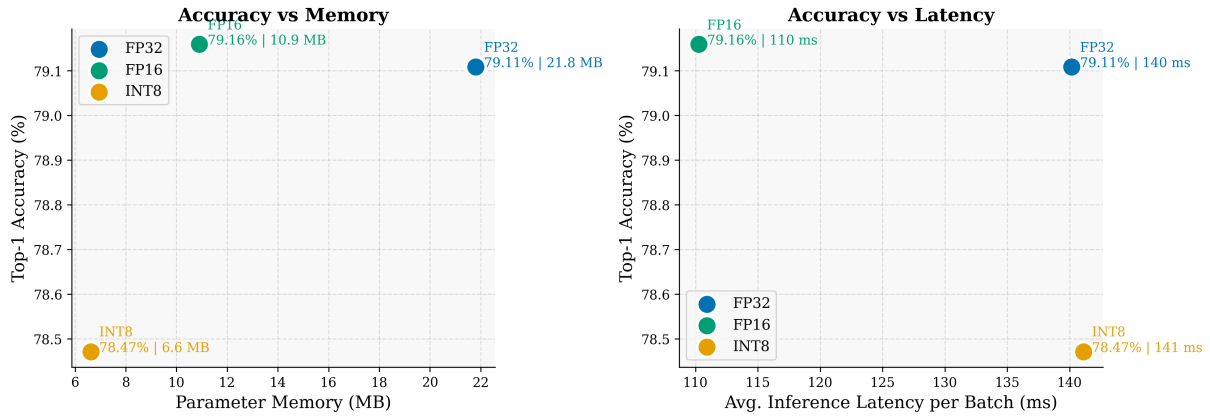
## 6 Comparație Globală: FP32 / FP16 / INT8

Comparație FP32 / FP16 / INT8 — ViT-Tiny pe ImageNette  
(FP16 = model.half() | INT8 = per-tensor weight-only)



**Figura 6:** Comparație directă FP32 / FP16 / INT8 pe cele trei metrice: acuratețe, memorie și latență. FP16 domină: memorie 2× mai mică și latență 1.28× mai mică, cu degradare nulă. INT8 oferă cea mai mare compresie de memorie (3.29×), dar fără beneficiu de viteză pe acest hardware.

Accuracy-Efficiency Tradeoff — FP32 / FP16 / INT8



**Figura 7:** *Stânga:* Scatter accuracy vs. memorie. INT8 oferă cel mai bun raport memorie/acuratețe. *Dreapta:* Scatter accuracy vs. latență. FP16 domină clar: acuratețe identică cu FP32, dar latență mai mică.

**Tabela 6:** Tabel comparativ complet FP32 / FP16 / INT8

| Metodă     | Acc.   | $\Delta$ Acc. | Loss   | Latență | Mem.     | Compresie |
|------------|--------|---------------|--------|---------|----------|-----------|
| FP32       | 79.11% | —             | 0.7474 | 141 ms  | 21.81 MB | 1×        |
| FP16       | 79.16% | −0.05 pp      | 0.7472 | 110 ms  | 10.91 MB | 2.00×     |
| INT8 (w-o) | 78.47% | +0.64 pp      | 0.7712 | 141 ms  | 6.62 MB  | 3.29×     |

*w-o = weight-only; latența este per batch (bs=64) pe Apple M4 MPS.*

## 7 Analiză și Discuții

### 7.1 FP16 — conversie fără cost

Conversia `float32`  $\rightarrow$  `float16` nu introduce nicio degradare practică de acuratețe ( $-0.051$  pp, în limitele zgomotului numeric). Câștigurile sunt:

- **Memorie:**  $2\times$  reducere exactă, conformă cu teoria ( $4$  bytes  $\rightarrow$   $2$  bytes per parametru)
- **Latență:**  $1.28\times$  speedup real pe MPS (Apple M4 valorifică `float16` nativ, fără unități Tensor Core dedicate)
- **Compatibilitate MPS completă:** `model.half()` funcționează fără excepții pe PyTorch 2.9, inclusiv LayerNorm

### 7.2 INT8 weight-only — compresie maximă fără speedup

Cuantizarea INT8 per-tensor oferă cea mai mare compresie de memorie ( $3.29\times$ ), cu o degradare de acuratețe acceptabilă ( $+0.637$  pp). Însă **nu există speedup** față de FP32 deoarece implementarea dequantizează la `float32` la fiecare forward pass.

Outlierul identificat (`blocks.7.mlp.*`,  $\text{MSE} \approx 40\times$  media) sugerează că o strategie *per-channel* ar reduce semnificativ eroarea de cuantizare pe aceste straturi specifice — aceasta este una din analizele planificate pentru Faza 3.

### 7.3 Implicații pentru deployment

- **Pe Apple M4 (MPS):** FP16 este metoda optimă — fără pierdere de acuratețe,  $1.28\times$  mai rapid,  $2\times$  mai puțin memorie.
- **Pe NVIDIA (CUDA):** INT8 cu kernel-uri GEMM native (Tensor Cores) ar oferi atât compresie de memorie ( $3-4\times$ ) cât și speedup real ( $2-4\times$ ) față de FP32.
- **FP16 vs. INT8:** FP16 câștigă pe acuratețe și latență (pe hardware cu suport nativ), INT8 câștigă pe memorie — decizia depinde de constrângerea dominantă a aplicației.

## 8 Limitări ale studiului

- **Dataset:** ImageNette are doar 10 clase “ușoare” — rezultatele pot să nu se generalizeze la sarcini mai dificile (detectie, segmentare).
- **Hardware:** Apple M4 MPS nu are suport INT8 GEMM nativ — concluzia privind lipsa speedup-ului INT8 este specifică acestei arhitecturi.
- **Weight-only:** Activările rămân în `float32` — cuantizarea completă (weights + activations) ar putea oferi compresia deplină așteptată teoretic.
- **Per-tensor:** Granularitatea per-tensor este suboptimală pentru straturile cu distribuții de ponderi non-uniforme (`blocks.7.mlp.*`).

## 9 Concluzii și Pași Următori

### 9.1 Concluzii

1. **FP32 → FP16 (static):** Degradare nulă ( $-0.051$  pp),  $2\times$  mai puțin memorie,  $1.28\times$  mai rapid pe MPS. **Recomandare: utilizat implicit pentru deployment pe Apple Silicon.**
2. **FP32 → INT8 (weight-only, per-tensor):** Degradare acceptabilă ( $+0.637$  pp),  $3.29\times$  mai puțin memorie, fără speedup real pe MPS din cauza overhead-ului de dequantizare.
3. **Outlieri identificați:** Straturile `blocks.7.mlp.fc1` și `blocks.7.mlp.fc2` au MSE de cuantizare  $\approx 40\times$  mai mare decât media — candidați prioritari pentru per-channel quantization.
4. **Preprocessing timm:** Normalizarea corectă trebuie obținută din `timm.data.resolve_data_conf` — nu din constante hardcoded (diferența: 79% vs. 47% acuratețe cu normalizare greșită).

### 9.2 Faza 3 — Experimente planificate

- **Per-channel vs. per-tensor INT8** — compararea granularităților
- **Sensitivity analysis** — cuantizarea câte unui layer pe rând (impact individual per strat)
- **Layer-wise error** — MSE/MAE per strat, vizualizare heatmap
- **Memory footprint** — dimensiunea modelului serializat în fiecare format (`.pt`, `safetensors`)
- **Timing detaliat** — latență FP32 / FP16 / INT8 per-tensor / INT8 per-channel

## Bibliografie

- [1] A. Dosovitskiy, L. Beyer, A. Kolesnikov et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” *ICLR*, 2021. [arXiv:2010.11929](https://arxiv.org/abs/2010.11929).
- [2] H. Touvron, M. Cord, M. Douze et al., “Training data-efficient image transformers & distillation through attention,” *ICML*, 2021. [arXiv:2012.12877](https://arxiv.org/abs/2012.12877).
- [3] P. Micikevicius, D. Stosic, N. Burgess et al., “FP8 Formats for Deep Learning,” 2022. [arXiv:2209.05433](https://arxiv.org/abs/2209.05433).
- [4] B. Jacob, S. Kligys, B. Chen et al., “Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference,” *CVPR*, 2018. [arXiv:1712.05877](https://arxiv.org/abs/1712.05877).
- [5] J. Howard, “Imagenette,” fast.ai, 2019. <https://github.com/fastai/imagenette>.
- [6] R. Wightman, “PyTorch Image Models (timm),” 2019. <https://github.com/rwightman/pytorch-image-models>.
- [7] PyTorch Team, “Automatic Mixed Precision package — torch.amp,” Documentație oficială PyTorch, 2023. <https://pytorch.org/docs/stable/amp.html>.